Problem List | Submit | Premium

Description | Editorial | Solutions | Submissions

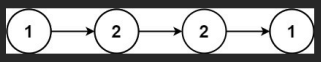
234. Palindrome Linked List

Solved

Easy | Topics | Companies

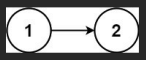
Given the `head` of a singly linked list, return `true` *if it is a palindrome or* `false` *otherwise.*

Example 1:




```
Input: head = [1,2,2,1]
Output: true
```

Example 2:




```
Input: head = [1,2]
Output: false
```

Constraints:

- The number of nodes in the list is in the range `[1, 10^5]`.
- `0 <= Node.val <= 9`

Follow up: Could you do it in `O(n)` time and `O(1)` space?

PMI Apply Now

18.7K | 447 | 59 Online

Code

C++ | Auto

```cpp
class Solution {
public:
    bool isPalindrome(ListNode* head) {
        if (head == nullptr || head->next == nullptr) {
            return true;
        }

        // Step 1: Find the middle
        ListNode* slow = head;
        ListNode* fast = head;

        while (fast != nullptr && fast->next != nullptr) {
            slow = slow->next;
            fast = fast->next->next;
        }

        // Step 2: Reverse the second half
        ListNode* prev = nullptr;
        ListNode* curr = slow;

        while (curr != nullptr) {
            ListNode* nextNode = curr->next;
            curr->next = prev;
            prev = curr;
            curr = nextNode;
        }

        // Step 3: Compare both halves
        ListNode* left = head;
        ListNode* right = prev;

        while (right != nullptr) {
            if (left->val != right->val) {
                return false;
            }

            left = left->next;
            right = right->next;
        }
```

probal2006
Access all features with our Premium subscription!
My Lists | Notebook | Progress | Points
Try New Features
Orders
My Playgrounds
Settings
Appearance
Sign Out

Restored from local | Upgrade to Cloud Saving | Ln 1, Col 1

Testcase | Test Result

Problem List | Submit | Premium

Description | Editorial | Solutions | Submissions

328. Odd Even Linked List

Solved

Medium | Topics | Companies

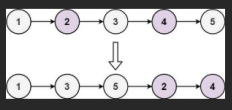
Given the `head` of a singly linked list, group all the nodes with odd indices together followed by the nodes with even indices, and return *the reordered list.*

The **first** node is considered **odd**, and the **second** node is **even**, and so on.

Note that the relative order inside both the even and odd groups should remain as it was in the input.

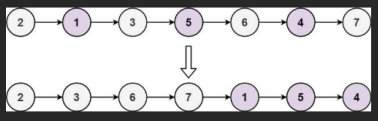
You must solve the problem in `O(1)` extra space complexity and `O(n)` time complexity.

Example 1:




```
Input: head = [1,2,3,4,5]
Output: [1,3,5,2,4]
```

Example 2:




```
Input: head = [2,1,3,5,6,4,7]
Output: [2,3,6,7,1,5,4]
```

11.8K | 264 | 48 Online

Code

C++ | Auto

```cpp
class Solution {
public:
    ListNode* oddEvenList(ListNode* head) {
        if (head == nullptr || head->next == nullptr) {
            return head;
        }

        ListNode* odd = head;
        ListNode* even = head->next;
        ListNode* evenHead = even;

        while (even != nullptr && even->next != nullptr) {
            // Connect odd nodes
            odd->next = even->next;
            odd = odd->next;

            // Connect even nodes
            even->next = odd->next;
            even = even->next;
        }

        // Connect odd list with even list
        odd->next = evenHead;

        return head;
    }
};
```

probal2006
Access all features with our Premium subscription!
My Lists | Notebook | Progress | Points
Try New Features
Orders
My Playgrounds
Settings
Appearance
Sign Out

Restored from local | Upgrade to Cloud Saving | Ln 1, Col 1

Testcase | Test Result